AI Engineer Case Study — Northwind Logistics

Hello, and thank you for taking the time to work through this case study. We use it to understand how you think about an open-ended applied-AI problem end-to-end: how you frame it, how you choose tools, how you build, what you measure, and how you ship.

This brief intentionally does not tell you what to build. It describes the business problem, the materials we are giving you, and the outcomes we will test for. The architecture, the technology choices, the API surface, the user interface, the retrieval approach, the evaluation methodology, and the deployment target are all yours to design. Those design decisions are a large part of what we are evaluating.

Use whatever coding tools help you. AI pair programmers, copilots, code generators, autocomplete, snippet libraries — anything you'd reach for in real work. We care about what you build and why, not the keystrokes you typed to get there.

The business problem

Northwind Logistics is a fictional mid-size logistics company. Their finance team currently reviews employee expense submissions manually, comparing each line item against a sprawling library of company policies. The work is slow, inconsistent, and a recruiting headache.

The CFO wants an **AI-assisted pre-review system** that ingests an expense submission and surfaces what a reviewer needs to act on — what looks compliant, what looks like a policy violation and why, what's ambiguous and needs a human eye. A human reviewer always makes the final call; the system should do the heavy lifting and explain its reasoning so the reviewer can trust or override it.

What we are giving you

- `**policies/` ** — Northwind Logistics' policy library, ~30 PDF documents totaling roughly 100 pages. Most are about travel and expense; a meaningful fraction are unrelated corporate policies (data classification, vendor onboarding, business continuity, etc.) that exist as realistic noise. Cross-references between policies use document IDs (e.g., `TEP-002 §2.3`).
- `**submissions/**` — five sample expense submissions, each a folder containing:
- `employee_info.json` — the employee's identity, grade, manager, department, and the purpose and dates of the trip. These are **seed data**: load the five employees into your system on startup so they're available in your UI. Do not require reviewers to upload JSON in the browser.

- `receipts/` — 4 to 8 PDF receipts representing the employee's expenses on that trip. Each receipt is one line item. There is no separate expense report or "claim" — the receipts are the claim.

We will also test your system with **image-format (`.jpg` / `.png`) and plain-text (`.txt`) receipts** during grading. Don't assume PDFs only.

What the system needs to do (capabilities, not implementation)

A finance reviewer should be able to, from a normal browser:

1. **Start a new submission for an employee.** Either pick an existing employee from a list (seeded from the five we provide) or create a new one with the needed identity and trip context.
2. **Upload receipts** in mixed formats (PDF, image, text) and have the system extract what's needed from each.
3. **See the system's pre-review of every line item** — what category it is, what verdict the system reached (compliant, flagged, rejected, or however you choose to express it), the system's reasoning, the policy clause(s) it relied on (quoted, not just referenced), and how confident it is. Flagged items must be visually distinct from compliant ones.
4. **Override any verdict** with a comment, and have that override persist and be auditable.
5. **Browse the history of past submissions** — by employee, by date, by status — and click into any of them to see the original verdicts and any applied overrides. State must persist across server restarts; in-memory is not acceptable.
6. **Ask ad-hoc questions** about the policy library and receive cited, grounded answers — and have the system decline questions outside the policy library rather than fabricate.

How you structure the UI, what views you build, how you partition the backend, how you do retrieval, what model you use, where you persist data — all up to you.

Sample submissions

The five sample submissions span a deliberate range. Some are clean. Some contain real policy violations of different types. Some require the system to understand the employee's *trip context* — not just the receipt content — to reach the right verdict. We will tell you only after submission which were which; part of the evaluation is whether your system produces sensible, well-explained verdicts on all of them.

Deliverables

1. A **public GitHub repository** with your code.
2. A **live deployed URL** we can open and use in a browser.
3. A **README** in the repo that:

4. Tells us how to run it locally (and what API keys, if any, we'd need to supply)
5. Describes your architecture, with a diagram or sketch
6. Explains the **why** behind your design choices, especially anywhere you faced a real tradeoff (retrieval approach, chunking strategy, model tier selection, when to use a vision model, how to handle confidence, where to draw the line on what to flag vs reject vs ask a human)
7. Reports rough cost per submission and how you'd scale to 10,000 submissions per day
8. Notes what you'd do next if you kept working on this
9. An **evaluation harness** — a script we can run that exercises your system against a held-out set we'll provide after submission. Design the harness so we can drop in a JSON file of expected outcomes and get back numbers we care about (accuracy, retrieval quality, citation correctness, refusal rate on out-of-scope queries). What "the right metrics" are is part of your design — show us what you chose to measure and why.

How we evaluate

We will:

- **Use your deployed system in the browser**, walking through the capabilities above with the sample submissions and with additional test material of our own.
- **Read your code**, looking at how you structured the pipeline, whether you used schema-constrained LLM outputs or parsed free text, how you handled failure modes, and whether the code is readable.
- **Run your eval harness** against our held-out set.
- **Read your README**, looking for clear thinking about tradeoffs — not just descriptions of what you built.

The strongest signal for us is **an end-to-end system that demonstrably works**, paired with **honest reasoning about tradeoffs** in the README. The weakest signals are: a great backend with no UI; a great UI on top of a non-working backend; a system that confidently produces wrong answers without acknowledging uncertainty; a README that lists features rather than defending decisions; and no evaluation methodology at all.

A few things we want to call out explicitly

- **No expected scoreboard.** Don't optimize for a specific benchmark we haven't shown you. Optimize for a system that handles the messiness of real receipts and real policies.
- **Honest "I don't know."** A system that refuses to answer or marks an item low-confidence when retrieval is weak is worth more than one that confidently picks a wrong answer. We will deliberately test this.
- **Citation faithfulness.** If your system says "policy TEP-X says Y," the actual quoted clause must support Y. We will spot-check.

- **Persistence, not in-memory.** A submission processed yesterday should be visible today after a server restart.
- **You're allowed to pick tools we wouldn't.** If you make an unusual choice, defend it in the README. We don't need you to use any specific framework, vector store, model provider, or deployment target.

Good luck — looking forward to seeing what you build.